

LJA — Data & Algorithm Pipeline

Learning Journey Assistant — Data and Algorithm Pipeline

How a mark becomes a recommendation

What the gap engine does

Where the Moodle score comes from

Trust boundaries

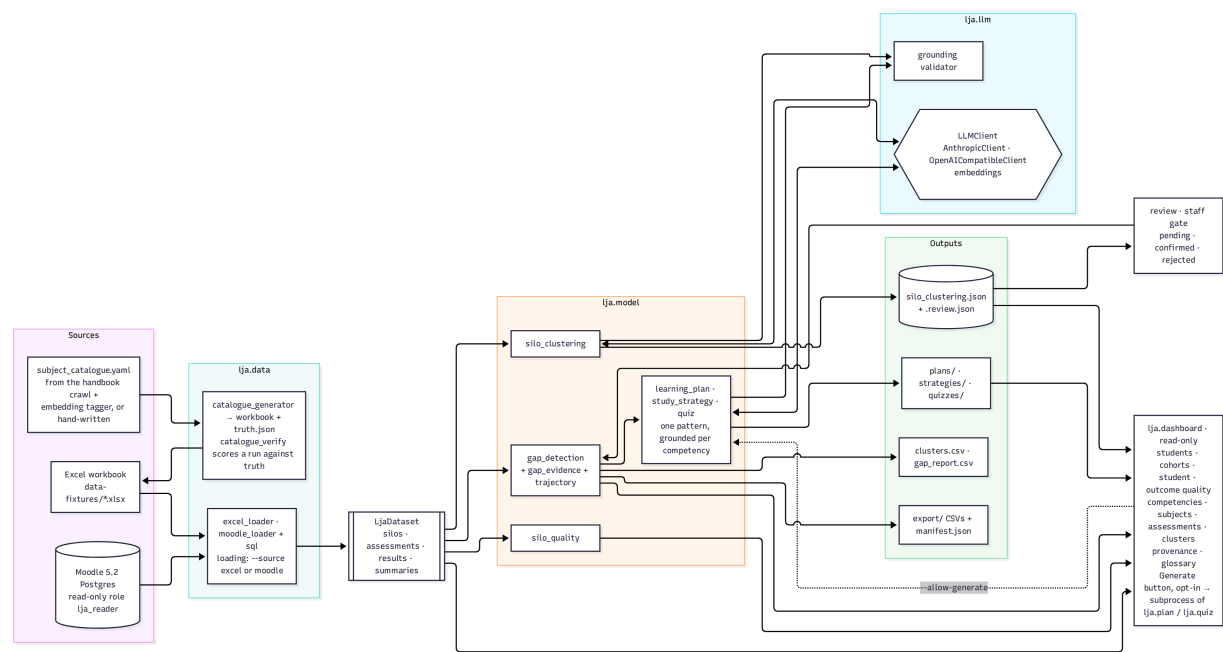
What changed since the August page

Learning Journey Assistant — Data and Algorithm Pipeline

Version	2.0, 3 October 2026. Supersedes the 11 August 2026 page, which described the target design before any of it ran.
Describes	main at b4ccebdc (3 Oct 2026), after pull requests #52 to #54 and #25; the opt-in Generate button is in open PR #55
Source	docs/design/data-and-algorithm-pipeline.md; diagrams rendered from docs/handover/diagrams/*.mmd. Rebuild with docs/design/build.sh
Companions	CLI Pipeline (what one <code>lja.cli</code> run does), <i>UML Use Case and Sequence Diagrams</i> , the System Maintenance Document (the full reference) and the User Document (how to read the dashboard)

How a mark becomes a recommendation

Seven stages, one algorithmic core. Everything before gap detection is plumbing that produces one in-memory dataset; everything after it is presentation or generation that reads the gap engine's verdicts. The language model is allowed to see structured data only, and every name it returns is checked against that data before anything is written.



LJA architecture

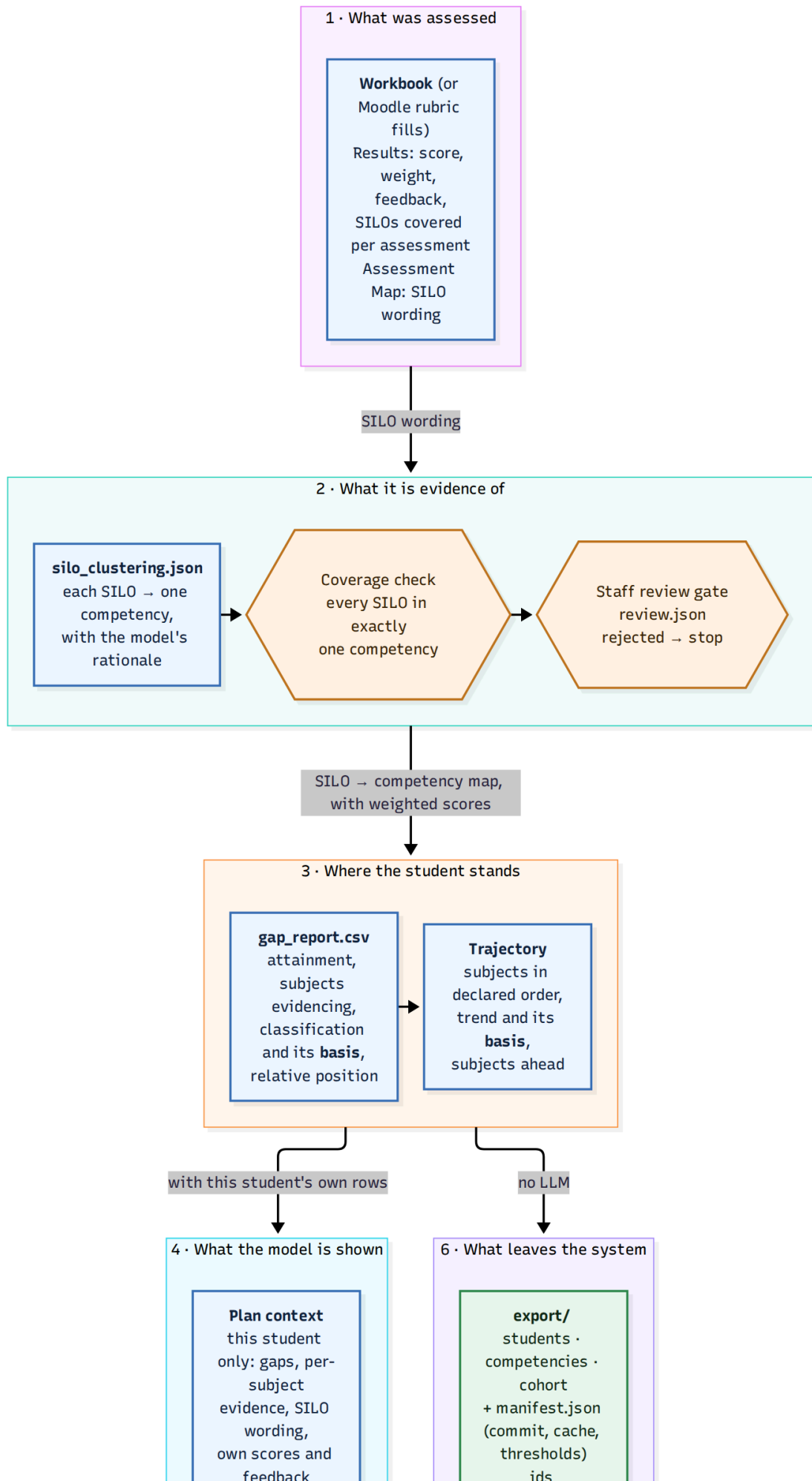
Figure 1: Sources, data layer, the `LjaDataset` interchange type, model layer, LLM layer, outputs and dashboard.
Source: docs/handover/diagrams/01-architecture.mmd.

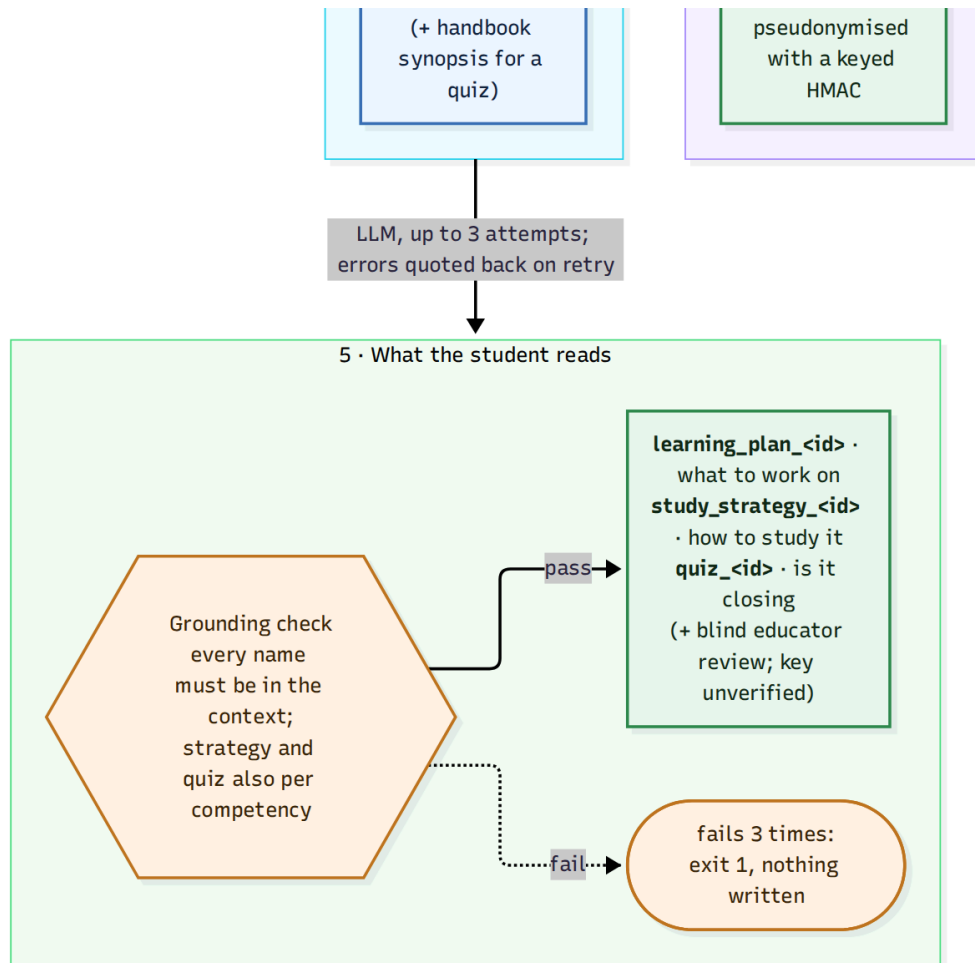
Stage	Owning code	Input	Output	Status, 3 Oct 2026
Sources	data-fixtures/, devenv/, lja.data.handbook, lja.data.catalogue_generator	The owner's workbook; a Moodle 5.2 database; the public La Trobe handbook; a subject catalogue	A workbook, or rubric fills, or a synthetic cohort with an answer key	Working. Moodle proven for one seeded subject with read-only enforcement observed (IOLG-111). The handbook crawl and the 100- subject, 5,000-student cohort are reproducible but not committed.

Stage	Owning code	Input	Output	Status, 3 Oct 2026
Extraction	<code>lja.data.excel_loader</code> , <code>lja.data.moodle_loader</code> + <code>sql/</code> , <code>lja.data.loading</code>	Workbook sheets, or SQL Query 2 over the rubric tables plus a staff- edited criterion- to-SILO map	One <code>LjaDataset</code> : SILOs, assessments, result rows, student summaries	Working. Both loaders produce the same type; <code>--source excel</code> or <code>moodle</code> on every command.
Clustering	<code>lja.model.silo_clustering</code> , <code>lja.llm</code>	SILO wording	Competency clusters with a label and rationale, cached as <code>silo_clustering.json</code>	Working with coverage validation and retry. Single-call clustering fails above about 50 SILOs; the embedding tagger in <code>lja.data.competency_tagger</code> is the staged answer (ADR 0002 trigger hit).
Staff gate	<code>lja.review</code>	The cache	<code>.review.json</code> with pending / confirmed / rejected per cluster	Working, CLI only. A rejected cluster stops the pipeline; pending needs <code>--allow-unconfirmed</code> ; the dashboard shows a banner.
Gap detection	<code>lja.model.gap_detection</code> , <code>gap_evidence</code> , <code>trajectory</code>	Dataset plus confirmed clusters	<code>gap_report.csv</code> : attainment, classification, its basis, relative position; per-subject evidence; trend along the declared subject sequence	Working. Relative to the student's own profile (median and MAD) with absolute guards; every threshold is configuration and unratified (A-01).
Generation	<code>lja.model.learning_plan</code> , <code>study_strategy</code> , <code>quiz</code> ; <code>lja.llm.grounding</code>	This student's gaps, evidence, SILO wording, own scores and feedback	<code>learning_plan_<id></code> , <code>study_strategy_<id></code> , <code>quiz_<id></code> , JSON and Markdown, written only after grounding passes	Working. Plans and strategies reliable; the quiz is a thin slice whose answer key cannot be verified, and it fails for students with many gaps on the local model (per-competency generation is the follow-up).
Presentation and export	<code>lja.dashboard</code> , <code>lja.export</code>	Dataset, gaps, cache, generated files	Students, cohorts and priority groups; student page with gaps, strengths, progress, subject chain, plan, quiz; outcome quality, competencies, subjects, assessments, clusters; provenance; glossary. Three research CSVs and a manifest, pseudonymised	Working. The dashboard never calls a model on a page load; the opt-in Generate button (PR #55) runs the CLI as a subprocess.

What the gap engine does

The owner's primary signal is variability within a student's own profile, not raw failure. For each student and competency the engine computes a weighted attainment, then classifies it in a fixed order: below the absolute floor is a gap; at or above the absolute ceiling is proficient; a profile with too few competencies or too flat a spread falls back to an absolute split and says so; otherwise the position in MAD units below or above the student's own median decides. A gap evidenced in two or more subjects is persistent, otherwise isolated. Every row carries the rule that decided it, so no displayed figure is unexplained.





Evidence lineage

Figure 2: What each stage leaves behind and the check between stages, so any statement a student reads can be followed back to marked work. Source: `docs/handover/diagrams/07-evidence-lineage.mmd`.

Where the Moodle score comes from

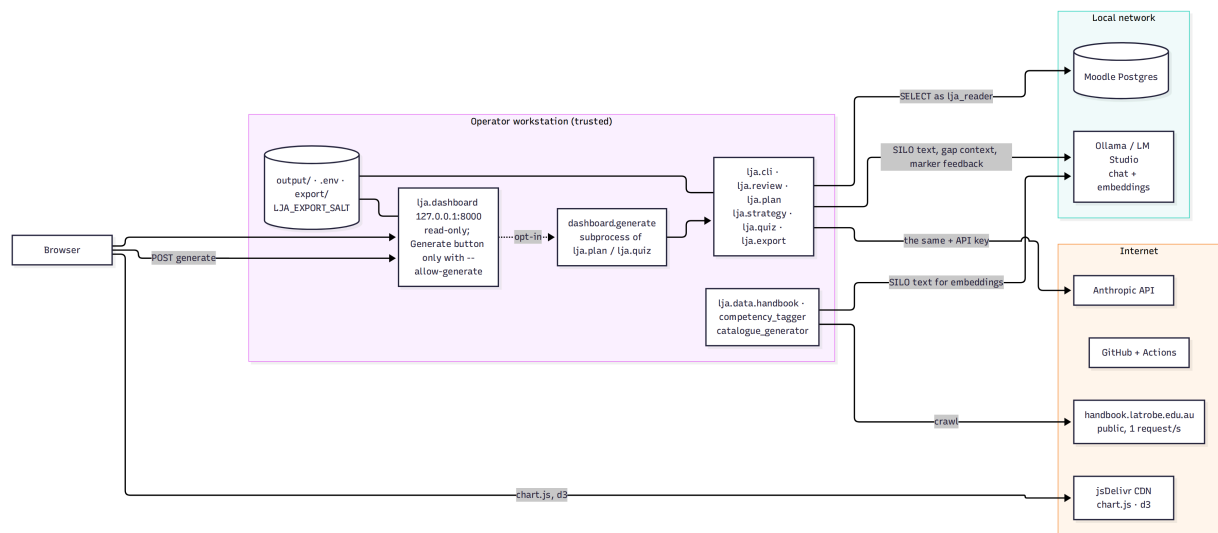
Only the direct-SQL path can reach a rubric *filling*, the level a marker chose and their remark. Moodle’s Web Services expose a rubric’s definition but no function returns what was awarded. So the production path connects to PostgreSQL as the `lja_reader` role, which can only `SELECT`, and runs Query 2 in `sql/moodle_attainment_extraction.sql`. The loader joins those rows to a staff-editable `criterion_silo_map_<SUBJECT>.csv`; an unmapped criterion is a hard error, not a silent drop.

Moodle field	What it holds	Becomes
<code>gradingform_rubric_fillings.remark</code>	Marker’s comment on this one criterion	<code>feedback_comment</code>
<code>grading_instances.status</code>	Must equal 1 (active); 0, 2, 3 are stale, draft or superseded	filter only
<code>grading_instances.itemid</code>	Not a user id: joins to <code>assign_grades.id</code> , the common wrong join in this schema	join key
<code>gradingform_rubric_criteria.description</code>	The criterion’s own text	mapped to a SILO key via the CSV
<code>gradingform_rubric_levels.score</code> and the criterion’s maximum	Points awarded, and the ceiling	<code>score</code> , normalised to a percentage
<code>assign.name</code> , <code>course.shortname</code> , <code>user.idnumber</code>	Assessment, subject, student	<code>assessment_name</code> , <code>subject_code</code> , <code>student_id</code>

The 11 August design had a staging table named `lja_criterion_score` and a `confirmed_by_staff` gate on the criterion map. Neither was built that way. The in-memory `LjaDataset` is the staging table, and the staff gate moved one stage later, onto the LLM’s clustering, which is where the uncertainty actually is.

What was observed on 28 September 2026 (IOLG-111, `docs/security-evidence.md`): the role created from the DDL in `sql/README.md`; `UPDATE m_user as lja_reader` refused with `permission denied`; a data-only dump byte-identical before and after a pipeline run.

Trust boundaries



Trust boundaries

Figure 3: What crosses each boundary. Prompts carry SILO text, gap context and the student's own marker feedback, never names beyond STU0001-style ids. Source: `docs/handover/diagrams/06-trust-boundaries.mmd`.

What changed since the August page

August 2026 said	October 2026
Extraction “partial, loader not coded”; mapping “schema only”	Both loaders working; the criterion map is a CSV, not a table
Gap detection “SQL drafted, not yet Python”, two fixed thresholds 50/65	Python, relative to the student’s profile with seven configurable thresholds; SQL Query 6 kept as an annotated legacy
LLM layer “planned”	Two providers behind one protocol; clustering, plans, strategies and quizzes all grounded by code
Dashboard “planned”, four views	Fourteen pages, every count a link, provenance and glossary; opt-in generation
No research export, no synthetic data, no handbook	<code>lja.export</code> with pseudonyms; catalogue generator with planted gaps and an answer key; handbook crawl with an embedding tagger
Staff review gate on the criterion map	Gate on the clustering, with a dashboard banner and a CLI stop